Test Plan

One section per hard problem, in the same order as the design document. For each: what the suite asserts, and — the part that matters — how each test fails if the design is wrong in the way I am most worried about. All three problems now have real pytest files (tests/), running against a live in-process Iceberg catalog: Spark local[2], hadoop catalog type, a tmp warehouse, iceberg-spark-runtime pinned in conftest.py. The assertions are against real MERGE, real branch DDL and real fast_forward semantics, not mocks of them, which is the only way a test can catch a wrong API rather than a wrong idea.

Two notes on what I changed after reviewing the first version of this plan against the tests it described. It claimed the P0 path was covered by test_main_is_untouched_while_held; it was not — that test held on the watermark and never executed a check, so nothing in the suite exercised a P0 failure at all. And it had no test for hard problem 3 on the argument that the other two were harder, which was the wrong thing to skip, because the cheapest test in the suite guards the difference between a redelivered settlement file and a doubled day of revenue.

1. CDC replay safety (dataplatform/cdc_apply.py)

The suite deliberately separates the two mechanisms, because they carry different weight and the earlier version conflated them: the LSN predicate makes a replay correct, the marker table only makes it cheap. So the load-bearing test removes the marker entirely and still demands correct state.

- **test_replay_after_lost_marker_leaves_state_unchanged —** the crash case. The marker write raises after the MERGE has landed; the identical offset range is then replayed with no marker to short-circuit on, and observable state must be unchanged. An extra Iceberg commit is accepted and asserted as such; a changed row is not.
- **test_replay_with_marker_writes_no_new_snapshot —** the happy path. A marked batch is skipped outright, writing no second data commit and no duplicate marker.
- **test_stale_batch_cannot_revert —** an older source LSN can never overwrite a row already updated by a newer one, regardless of arrival order.
- **test_delete_is_ordered_like_any_other_version —** a tombstone obeys the same ordering guard as an update, so a late stale insert cannot resurrect a deleted key, and the key stays single-rowed.
- **test_in_batch_collapse_keeps_the_newest_version —** several changes to one key inside one batch collapse to the newest, not the oldest — the difference between a correct table and one that silently lags by a batch.
- **test_batch_id_is_pure_and_range_order_insensitive —** no Spark required: the id is stable across calls, insensitive to the order the partition ranges are listed in, and distinct for a different range or topic. If this drifts, every short-circuit above is void.

How each fails if the design were wrong:

- **Drop the s._lsn > t._lsn predicate —** test_stale_batch_cannot_revert and test_replay_after_lost_marker_leaves_state_unchanged both fail. This is the scenario that corrupts a real balance, and it is the one nothing else in the system would notice.
- **Change > to >= —** a same-version reapply now rewrites the row, so the replay-after-lost-marker test fails on data_snapshot_count. Note that it does not fail on state, which is exactly why the snapshot assertion is there.
- **Make batch_id() impure —** add a wall-clock or an attempt counter and the replay mints a fresh id, the probe misses, and test_replay_with_marker_writes_no_new_snapshot fails.
- **Mark the batch applied before the MERGE instead of after —** a crash in between would then skip a batch that never landed. This is the failure that loses data rather than duplicating work, and I want to be straight that it is not covered: the ordering is not injectable, so it is asserted by reading apply_micro_batch. test_foreign_marker_does_not_suppress_a_batch is the weaker thing I could actually test.
- **Physically delete on a Debezium 'd' —** the stale insert no longer matches a target row and is inserted as live, so test_delete_is_ordered_like_any_other_version fails on both the flag and the row count.

2. The publish gate (dataplatform/publish_gate.py)

- **test_stalled_upstream_is_held —** healthy loan feed, repayment feed three days behind: HELD, and the candidate branch is cleaned up rather than left behind.
- **test_hole_in_upstream_history_is_held —** the subtle one, and the case the first version of this suite missed entirely. payments_core has reported every date up to D except D-2. max(business_date) says D and publishes a close built on a missing day; the watermark has to say D-3.
- **test_unknown_upstream_is_not_treated_as_current —** an upstream that has never reported yields an unknown watermark, not "whatever date happens to be latest", and still resolves to HELD.
- **test_p0_failure_holds_and_main_is_untouched —** every upstream complete, so the gate actually reaches the check suite, and the data carries a negative paise value. HELD, main's row count unchanged, valid_through null, and the findings column is asserted to be valid JSON — str(dict) emits single quotes and would corrupt the manifest row it was supposed to explain.
- **test_publishes_once_the_laggard_catches_up —** once the laggard catches up, the same partition publishes, valid_through is set, and the branch is cleaned up. The gate must not be a one-way ratchet that stays stuck after the underlying problem is fixed.
- **test_override_publishes_incomplete_data_but_says_so —** the escape hatch works and is on the record: status OVERRIDDEN rather than PUBLISHED, with the approver and ticket in the reason.

- **test_override_cannot_waive_a_p0 —** the same override against data with a negative principal is refused. Incompleteness is a judgement call, a broken money column is not, and that line is policy, so it gets a test rather than a comment.
- **test_valid_through_drops_back_when_a_published_date_is_later_held —** a date that published on Monday and is held on Tuesday stops being served, because the serving view reads the latest decision per date rather than the existence of a publish.

How each fails if the design were wrong:

- **effective_watermark takes max instead of min —** test_stalled_upstream_is_held fails at the HELD assertion: the stalled feed is ignored and the mart publishes on the healthy feed's schedule, which is precisely the plausible-but-wrong output this problem exists to prevent.
- **source_watermark reverts to max(business_date) —** test_hole_in_upstream_history_is_held fails. Same class of failure as above, one level down and much easier to ship without noticing, because the newest date really is complete.
- **Missing coerced to today —** test_unknown_upstream_is_not_treated_as_current fails: silence from an upstream becomes indistinguishable from that upstream being fully current.
- **Run checks against main, or fast_forward before checking —** test_p0_failure_holds_and_main_is_untouched fails, and it fails exactly when a consumer would otherwise see a partition the gate meant to refuse.
- **Hardcode the catalog in the fast_forward call —** test_publishes_once_the_laggard_catches_up and the two override tests all fail on the catalog name. Worth stating because the earlier version of this module hardcoded one catalog and every gate test would have errored on setup rather than on a claim.

The fixtures monkeypatch paging and warning to no-ops, so the assertions are on the manifest and on main's row count — the consumer-visible contract — rather than on whether a page fired, which is an implementation detail of the alerting integration. The mart is seeded with an earlier partition so "main is untouched" runs against a non-empty baseline: against zero rows, a bug that writes nothing would also pass.

3. Restatement and knowledge time (dataplatform/restatement.py)

- **test_renamed_but_identical_file_is_a_duplicate —** same bytes under a new name is a no-op. Partners rename and re-zip constantly, and only the bytes are evidence.
- **test_redelivery_while_first_load_is_in_flight_is_a_duplicate —** the bug the first version of classify actually had. The first claim is CLAIMED, not yet COMPLETE; a lookup restricted to COMPLETE returns nothing, the redelivery reads as brand new, and the same file loads twice concurrently into the same partition.
- **test_new_content_for_a_known_date_is_a_restatement —** same date, different bytes is a correction and never an append — the default behaviour of every naive file loader, and the one that silently doubles a day's settlement totals.

- **test_restatement_takes_the_next_knowledge_version —** versions count forward across successive corrections rather than resetting or colliding.
- **test_second_worker_does_not_own_the_claim —** two workers see the same drop and exactly one owns it. The loser's MERGE is a legitimate no-op and therefore does not raise, so "did not raise" is worthless as proof of ownership; the test asserts on the worker_id read back and on there being exactly one ledger row.
- **test_tag_survives_ordinary_snapshot_expiry —** the audit story end to end: cut a tag, write more, expire every ordinary snapshot, and read the tag. If RETAIN were missing or in the wrong unit the expiry would drop the snapshot the tag points at and the read would fail instead of returning what we believed in March.

Two things here are not tested, and named rather than buried. First, the composition itself: publish_restatement gates the correction and only then cuts the tag and emits the invalidations, which is what makes G8 and G9 true rather than aspirational, and it has no test — the pieces are each covered, the wiring between them is not. Second, and harder: that a reader querying the table mid-overwrite never observes the partition as empty or doubled. It depends on Iceberg's commit semantics behaving exactly as documented under concurrent snapshot creation, which the design assumes rather than verifies, and provoking it deterministically needs a concurrent reader thread racing a writer — a test I would want to write against a real cluster rather than local[2], where the scheduler makes the interleaving I care about unlikely to occur.

Coverage note

All three files test behaviour through the public entry points — apply_micro_batch, publish_partition, classify and claim — rather than internal helpers, because the guarantees being defended are properties of that boundary. A helper could be refactored entirely and these would still be the right tests to have. The one place I broke that rule is test_batch_id_is_pure_and_range_order_insensitive, which pins an internal detail deliberately: purity is not observable from the outside, and every replay short-circuit depends on it.